DSCI 519 · Homework 3

Cleaning a Conference Registration List with Contracts and a Hybrid Cleaner

Covers: Session 3 (Chapter 6). Points: 100. Submit: notebook with outputs, the PDF made by the last cell, and `hw3_clean_registrations.csv`.

The situation

The Midwest Analytics Summit (a fictional regional conference) exported 120 registrations from three sign-up forms that were built by different volunteers. The organizers need a clean attendee list for badges, catering, workshop room sizing, and revenue reconciliation. Some people registered twice. You have a labeled sample of 25 cleaned rows (`summit_dev_gold.csv`); the instructor will score your full output on the rest.

Target rules (what "clean" means)

Field	Clean value
`full_name`	`First Last` in title case ("SMITH, Ann" becomes "Ann Smith"; keep apostrophes such as O'Neil)
`email`, `email_valid`	lowercase email. Valid means one `@`, a name before it, and a domain with at least one dot after it (no spaces, no trailing dot). If the email is invalid, `email` is empty and `email_valid` is False
`phone`	`+1` and 10 digits, or empty
`company`	the company's name in normal capitalization; if it appears in several spellings, use the most frequent spelling that is not ALL CAPS; if every spelling is ALL CAPS (for example POET), keep it as written
`seniority`	Student (students, graduate assistants, PhD candidates), Individual Contributor (everyone not in another level), Manager (managers, team leads, supervisors, project managers), Director (directors, heads of a function), Executive (C-level, VP, SVP)
`state`	two-letter code (SD, ND, MN, IA, NE, WI, CO)
`registration_date`	`YYYY-MM-DD`; source dates are US month-first, and dates that start with the year are year, month, day
`ticket_type`	Early Bird (249), $Regular(349)$, Student (49), $Speaker(0)$
`amount_paid`	number in dollars ("free" and "comp" are 0); empty if missing
`dietary_category`	None, Vegetarian, Vegan, Gluten-Free, Halal, Kosher, Nut Allergy
`workshop`	LLMs for Analysts, dbt Fundamentals, Causal Inference, Dashboard Design, or None
`duplicate_of`	for a repeat registration, the `reg_id` of the person's first (lowest) registration; otherwise empty

Rubric

Part	What	Points
A	Data quality profile by dimension (rules)	10
B	Data contract with types, allowed values, and validators (including a cross-field rule)	20
C	Hybrid cleaner (deterministic first, AI only where judgment is needed) evaluated on the dev sample	35
D	Duplicate detection with verification and an audit log	15
E	Output file (scored on hidden labels), before/after quality report, reflection	20

Part E scoring: 10 points for the output file, scored on the hidden rows (average accuracy over the 13 cleaned columns; 95% or better earns all 10, 60% or less earns 0, linear in between; a missing or malformed file earns 0),

5 points for the before/after report, and 5 points for the reflection.

Where each technique is taught (Session 3 notebook): profiling with rules (1.1, and the dimensions in 0.4); contracts with `Literal`, `Optional`, `Field`, `field_validator` (0.1, 0.2, 2.1) and a cross-field `model_validator` (2.3); dates (0.5) and rule-based cleaning (4.1); `llm.extract` with failure handling (3.1, 4.2); the hybrid (4.4); duplicate keys and similarity (0.6), candidates, verification, and the audit log (4.5).

In [1]:
```
#@title Setup: install packages and the course helper (run me first)
# code hidden in this copy
```

```
llmkit 1.1.0 is ready.
```

Choose your AI provider

Everything in this course works with any of the providers below. Pick one, make sure its API key is stored in Colab Secrets (click the key icon in the left sidebar, add the secret name shown, and switch on *Notebook access*), and run the cell. You never paste a key into the notebook itself.

`PROVIDER`	Secret name	Cost notes (October 2026, verify before use)	Suggested `MIN_INTERVAL`
`"groq"`	`GROQ_API_KEY`	Free tier, no credit card, generous daily limits	2
`"google"`	`GOOGLE_API_KEY`	Free tier for Flash models (low daily quota; your data may be used for training)	6
`"openai"`	`OPENAI_API_KEY`	Paid, prepaid credit; small models cost cents for this course	0
`"anthropic"`	`ANTHROPIC_API_KEY`	Paid, prepaid credit; Haiku is the low-cost tier	0
`"openrouter"`	`OPENROUTER_API_KEY`	Many models behind one key; some `:free` models	3
`"mistral"`	`MISTRAL_API_KEY`	Free experimentation tier available	2
`"mock"`	none	Offline fake model: returns placeholders, useful only to test that code runs	0

`MODEL = None` uses the provider's default from `llmkit.PROVIDERS`. Model names change frequently; if you get a "model not found" error, run `llm.list_models()` and copy a current name into `MODEL`.

In [2]:
```
from llmkit import LLM, get_secret, estimate_tokens

PROVIDER = "google"      # "groq" | "google" | "openai" | "anthropic" | "openrouter" | "mistral" | "mock"
MODEL = None            # None = provider default, or a specific model id string
MIN_INTERVAL = 2.0      # seconds between calls (protects free-tier rate limits)

llm = LLM(PROVIDER, model=MODEL, min_interval=MIN_INTERVAL, cache=True)
print(llm)
```

```
Paste your GEMINI_API_KEY (input is hidden): ·····
LLM(provider='google', model='gemini-3.5-flash-lite')
```

In [3]:
```
#@title Get the data files
# code hidden in this copy
```

```
Please upload: ['summit_registrations_raw.csv', 'summit_dev_gold.csv']
```
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
```
Saving summit_dev_gold.csv to summit_dev_gold.csv
Saving summit_registrations_raw.csv to summit_registrations_raw.csv
Files present: ['summit_dev_gold.csv', 'summit_registrations_raw.csv']
```

In [4]:
```
import pandas as pd, numpy as np, re, json
raw = pd.read_csv("hw3/summit_registrations_raw.csv", dtype=str)
dev_gold = pd.read_csv("hw3/summit_dev_gold.csv", dtype=str, keep_default_na=False)   # keep the label "None" as text
print(raw.shape)
raw.head(10)
```

```
(120, 12)
```

Out[4]:

	reg_id	full_name	email	phone	company	job_title	state	registration_date	ticket_type	amount_paid	dietary_notes	workshop
0	R5068	Kara O'Neil	kara.oneil@outlook.com	(701) 892-6765	Cargill	Chief Data Officer	minnesota	January 15, 2026	EARLYBIRD	$249	NaN	Dashboard Design
1	R5090	Nikhil Wold	nikhil_wold@cargill.org	16058848469	Cargill	Business Analyst	south dakota	02/01/2026	Early Bird	$249.00	none	Dashboard Design
2	R5030	Emily Dvorak	emilyd@citibanksout.org	(515) 838-4412	Citibank South Dakota	Project Manager - Data	IOWA	2026-03-08	regular admission	USD 349	none	LLMs for Analysts
3	R5012	Ximena Solberg	ximena.solberg@southdakotas.org	NaN	South Dakota State University	MS Business Analytics Student	NE	2026-01-30	Student	$49	-	Causal Inference
4	R5027	Yusuf Iverson	yusufi@daktronics.org	701-797-7864	Daktronics	Reporting Specialist	ND	January 20, 2026	EARLYBIRD	USD 249	GF	NaN
5	R5015	KARA YODER	kyoder@southdakotas.org	605.671.7510	South Dakota State University	Undergraduate student	S.D.	January 19, 2026	student pass	$49	vegetarian	Dashboard Design
6	R5095	Pedro Quist	pquist@sanfordhealt.org	+1 605 899 1996	Sanford Health	Director of Analytics	SD	2026-01-23	regular admission	349.00	vegetarian	LLMs for Analysts
7	R5043	Elena Eide	elena_eide@yahoo.com	(605) 552-5270	SANFORD HEALTH	Data Science Team Lead	S Dakota	2026/03/20	Regular	$349.00	lacto-ovo vegetarian	Causal Inference
8	R5050	Lars Wold	larsw@yahoo.com	6058564913	South Dakota State University	Undergraduate student	South Dakota	2026-03-14	STUDENT	$49.00	-	LLMs for Analysts
9	R5060	Carmen Kowalski	carmen.kowalski@ravenindustr.org	16084966935	Raven Industries	Business Analyst	Iowa	02/19/2026	Early bird registration	NaN	no restrictions	Dashboard Design

---

Part A. Profile the data quality (10 points)

Write rule-based checks (pandas, regex) that count affected rows for at least six issues spanning at least four quality dimensions (completeness, validity, consistency, uniqueness, accuracy). Present a summary table, then list two issues your rules *cannot* detect and explain why. (Session 3, 1.1 builds a table of True/False checks and counts them with `.sum()`.)

In [5]:
```python
# Part A: rule-based data quality profile

# Helper: a column as stripped text, with blanks as ""
def col(c):
    return raw[c].fillna("").astype(str).str.strip()

# Reference values used by the rules
valid_states = {"SD", "ND", "MN", "IA", "NE", "WI", "CO"}
std_tickets = {"Early Bird", "Regular", "Student", "Speaker"}
std_diets = {"None", "Vegetarian", "Vegan", "Gluten-Free", "Halal", "Kosher", "Nut Allergy"}
std_workshops = {"LLMs for Analysts", "dbt Fundamentals", "Causal Inference", "Dashboard Design", "None"}
email_re = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"
iso_date_re = r"^\d{4}-\d{2}-\d{2}$"

# Turn an amount like "$249.00", "USD 349", "free" into a number
def amount_num(x):
    x = str(x).strip().lower()
    if x in ("free", "comp"):
        return 0.0
```

```
    digits = re.sub(r"[^\d.]", "", x)
    return float(digits) if digits else np.nan

# Expected price from the ticket label
def ticket_price(x):
    x = str(x).lower()
    if "early" in x:
        return 249.0
    if "student" in x:
        return 49.0
    if "speaker" in x:
        return 0.0
    if "regular" in x:
        return 349.0
    return np.nan

# Phone is valid if it has 10 digits, or 11 digits starting with 1
def phone_ok(x):
    d = re.sub(r"\D", "", x)
    return len(d) == 10 or (len(d) == 11 and d.startswith("1"))

# Prepare the text columns
name = col("full_name")
email = col("email").str.lower()
phone = col("phone")
company = col("company")
state = col("state")
date = col("registration_date")
ticket = col("ticket_type")
amount = col("amount_paid")
diet = col("dietary_notes")
workshop = col("workshop")

# Company spellings: flag rows whose company has more than one spelling
comp_key = company.str.lower()
n_spellings = company.groupby(comp_key).transform("nunique")

# Person key for duplicate names (word order and case ignored)
name_key = name.str.lower().str.replace(",", " ").str.split().apply(lambda w: " ".join(sorted(w)))

# Cross-field: amount paid does not match the ticket price
amt = amount.apply(amount_num)
price = ticket.apply(ticket_price)
price_mismatch = amt.notna() & price.notna() & (amt != price)

# Table of True/False checks (True = the row has the issue)
checks = pd.DataFrame({
    "email missing": email == "",
    "phone missing": phone == "",
    "amount_paid missing": amount == "",
    "job_title missing": col("job_title") == "",
    "email invalid format": (email != "") & ~(email.str.match(email_re) & ~email.str.endswith(".")),
    "phone not 10 digits": (phone != "") & ~phone.apply(phone_ok),
    "state not a valid 2-letter code": ~state.isin(valid_states),
    "date not YYYY-MM-DD": ~date.str.match(iso_date_re),
    "name not in First Last title case": name.str.contains(",") | name.str.isupper() | name.str.islower(),
    "company has several spellings": (company != "") & (n_spellings > 1),
    "ticket_type not a standard label": ~ticket.isin(std_tickets),
    "dietary_notes not a standard category": ~diet.isin(std_diets),
    "workshop not a standard name": (workshop != "") & ~workshop.isin(std_workshops),
    "duplicate email": (email != "") & email.duplicated(keep=False),
    "duplicate person name": name_key.duplicated(keep=False),
    "amount does not match ticket price": price_mismatch,
})

# Dimension for each check
```

```
dimension = {
    "email missing": "Completeness",
    "phone missing": "Completeness",
    "amount_paid missing": "Completeness",
    "job_title missing": "Completeness",
    "email invalid format": "Validity",
    "phone not 10 digits": "Validity",
    "state not a valid 2-letter code": "Validity",
    "date not YYYY-MM-DD": "Consistency",
    "name not in First Last title case": "Consistency",
    "company has several spellings": "Consistency",
    "ticket_type not a standard label": "Consistency",
    "dietary_notes not a standard category": "Consistency",
    "workshop not a standard name": "Consistency",
    "duplicate email": "Uniqueness",
    "duplicate person name": "Uniqueness",
    "amount does not match ticket price": "Accuracy",
}

# Summary table: rows affected per check
summary = pd.DataFrame({
    "dimension": [dimension[c] for c in checks.columns],
    "issue": checks.columns,
    "rows_affected": checks.sum().values,
})
summary["pct_of_rows"] = (summary["rows_affected"] / len(raw) * 100).round(1)
summary = summary.sort_values(["dimension", "rows_affected"], ascending=[True, False]).reset_index(drop=True)

print("Rows:", len(raw))
print("Rows with at least one issue:", checks.any(axis=1).sum())
print("Dimensions covered:", summary["dimension"].nunique())
summary
```

```
Rows: 120
Rows with at least one issue: 120
Dimensions covered: 5
```

Out[5]:

	dimension	issue	rows_affected	pct_of_rows
0	Accuracy	amount does not match ticket price	0	0.0
1	Completeness	phone missing	17	14.2
2	Completeness	amount_paid missing	2	1.7
3	Completeness	email missing	0	0.0
4	Completeness	job_title missing	0	0.0
5	Consistency	dietary_notes not a standard category	120	100.0
6	Consistency	company has several spellings	108	90.0
7	Consistency	date not YYYY-MM-DD	96	80.0
8	Consistency	ticket_type not a standard label	86	71.7
9	Consistency	name not in First Last title case	42	35.0
10	Consistency	workshop not a standard name	7	5.8
11	Uniqueness	duplicate email	20	16.7
12	Uniqueness	duplicate person name	18	15.0
13	Validity	state not a valid 2-letter code	94	78.3
14	Validity	email invalid format	9	7.5
15	Validity	phone not 10 digits	0	0.0

Two issues rules cannot detect, and why:

1. Emails that are well formed but wrong. Some emails pass the format check but have a misspelled or cut-off domain, such as `emilyd@citibanksout.org` (company is Citibank South Dakota) or

`pquist@sanfordhealt.org` (company is Sanford Health). A regex only checks the shape of the email (one @, a name, a domain with a dot). It has no way to know what the correct domain is, so these rows count as valid even though the email may never reach the person. Catching this needs outside knowledge, such as a list of real company domains or sending a test email.

2. The same person registered with different details. My uniqueness checks only flag exact matches after simple normalizing (same email, or the same name words). A person who registered twice with a different email (work vs. personal), a nickname, a missing apostrophe ("O'Neil" vs "ONeil"), or a typo in the name is not flagged. Deciding that two different-looking records are the same person needs fuzzy similarity plus judgment about several fields together (name, company, phone), which is what Part D does.

Part B. The data contract (20 points)

Define a Pydantic model `CleanRegistration` that encodes the target rules: `Literal` types for every categorical field, `Optional` where values may be missing, a `field_validator` for the date and one for the phone format, and a `model_validator` (cross-field rule) that rejects an `amount_paid` that does not match the ticket type's price (when the amount is present). Show it rejecting two hand-made bad records with readable error messages. Session 3, 2.1 shows a field validator and readable errors; 2.3 shows a `model_validator` for a cross-field price rule.

In [6]:
```python
# Part B: the contract
# Part B: the data contract
from typing import Literal, Optional
from datetime import datetime
from pydantic import BaseModel, Field, field_validator, model_validator, ValidationError

# Allowed values for the categorical fields
Seniority = Literal["Student", "Individual Contributor", "Manager", "Director", "Executive"]
State = Literal["SD", "ND", "MN", "IA", "NE", "WI", "CO"]
Ticket = Literal["Early Bird", "Regular", "Student", "Speaker"]
Diet = Literal["None", "Vegetarian", "Vegan", "Gluten-Free", "Halal", "Kosher", "Nut Allergy"]
Workshop = Literal["LLMs for Analysts", "dbt Fundamentals", "Causal Inference", "Dashboard Design", "None"]

# Price of each ticket type
PRICES = {"Early Bird": 249.0, "Regular": 349.0, "Student": 49.0, "Speaker": 0.0}

# The contract for one clean registration
class CleanRegistration(BaseModel):
    reg_id: str = Field(pattern=r"^R\d+$")
    full_name: str = Field(min_length=1)
    email: Optional[str] = None
    email_valid: bool
    phone: Optional[str] = None
    company: str = Field(min_length=1)
    seniority: Seniority
    state: State
    registration_date: str
    ticket_type: Ticket
    amount_paid: Optional[float] = Field(default=None, ge=0)
    dietary_category: Diet
    workshop: Workshop
    duplicate_of: Optional[str] = Field(default=None, pattern=r"^R\d+$")

    # Date must be a real date written as YYYY-MM-DD
    @field_validator("registration_date")
    @classmethod
    def check_date(cls, v):
```

```
        try:
            datetime.strptime(v, "%Y-%m-%d")
        except ValueError:
            raise ValueError(f"date '{v}' must be a real date in YYYY-MM-DD format")
        return v

    # Phone must be +1 followed by 10 digits, or missing
    @field_validator("phone")
    @classmethod
    def check_phone(cls, v):
        if v is None or v == "":
            return None
        if not re.fullmatch(r"\+1\d{10}", v):
            raise ValueError(f"phone '{v}' must be +1 followed by 10 digits")
        return v

    # Cross-field rule: amount paid must match the ticket price when present
    @model_validator(mode="after")
    def check_price(self):
        if self.amount_paid is not None:
            expected = PRICES[self.ticket_type]
            if self.amount_paid != expected:
                raise ValueError(
                    f"amount_paid {self.amount_paid:.0f} does not match the "
                    f"{self.ticket_type} price of {expected:.0f}"
                )
        return self

# Validate one record and print readable results
def check_record(label, record):
    try:
        CleanRegistration(**record)
        print(f"{label}: PASSED")
    except ValidationError as err:
        print(f"{label}: REJECTED")
        for e in err.errors():
            field = ".".join(str(p) for p in e["loc"]) or "record"
            msg = e["msg"].replace("Value error, ", "")
            print(f"   - {field}: {msg}")

# A good record to show the contract accepts clean data
good = {
    "reg_id": "R5068", "full_name": "Kara O'Neil", "email": "kara.oneil@outlook.com",
    "email_valid": True, "phone": "+17018926765", "company": "Cargill",
    "seniority": "Executive", "state": "MN", "registration_date": "2026-01-15",
    "ticket_type": "Early Bird", "amount_paid": 249, "dietary_category": "None",
    "workshop": "Dashboard Design", "duplicate_of": None,
}

# Bad record 1: wrong date format, bad phone, unknown state and workshop
bad1 = dict(good, reg_id="R9001", registration_date="01/15/2026",
            phone="(701) 892-6765", state="Minnesota", workshop="Excel Basics")

# Bad record 2: amount does not match the ticket price (cross-field rule)
bad2 = dict(good, reg_id="R9002", ticket_type="Student", amount_paid=349)

check_record("Good record", good)
check_record("Bad record 1", bad1)
check_record("Bad record 2", bad2)
```

```
Good record: PASSED
Bad record 1: REJECTED
   - phone: phone '(701) 892-6765' must be +1 followed by 10 digits
   - state: Input should be 'SD', 'ND', 'MN', 'IA', 'NE', 'WI' or 'CO'
   - registration_date: date '01/15/2026' must be a real date in YYYY-MM-DD format
   - workshop: Input should be 'LLMs for Analysts', 'dbt Fundamentals', 'Causal Inference', 'Dashboard Design' o
```

```
r 'None'
Bad record 2: REJECTED
   - record: amount_paid 349 does not match the Student price of 49
```

Part C. The hybrid cleaner (35 points)

Build a cleaner that produces a DataFrame `clean` with one row per raw registration and the columns in the target table (except `duplicate_of`, which is Part D). Requirements:

1. Deterministic first: names, emails, phones, companies, states, dates, ticket types, amounts, and workshops should be handled by code (mapping dictionaries, regex, `dateutil`).
2. AI where judgment is needed: use `llm.extract` with a small contract for seniority and dietary category (and, optionally, for any value your rules cannot resolve). Report the number of AI calls.
3. Validate every cleaned row against `CleanRegistration`; route failures to a review list instead of dropping them.
4. Evaluate field accuracy on the 25 dev rows and fix the largest error sources.

Tip: rows built with pandas store a missing amount as `NaN`, which fails `ge=0` in a Pydantic field. Convert NaN to `None` before validating (Session 3, 0.2).

In [7]:
```python
# Temporary: inspect llm.extract and the raw values before writing the cleaner
help(llm.extract)

# Gold sample: columns and a few rows
print(dev_gold.columns.tolist())
print(dev_gold.head(5).to_string())

# Every distinct raw value in the columns that need mapping
for c in ["state", "ticket_type", "amount_paid", "dietary_notes", "workshop", "job_title", "company"]:
    print("\n" + "=" * 50, c)
    print(raw[c].fillna("<blank>").value_counts().to_string())

# Sample of names, emails, phones, dates to see the formats
print(raw[["full_name", "email", "phone", "registration_date"]].sample(25, random_state=1).to_string())
```

```
Help on method extract in module llmkit:

extract(
    content: 'Any',
    schema: 'Type[Any]',
    instructions: 'Optional[str]' = None,
    max_repairs: 'int' = 2,
    **kwargs
) method of llmkit.LLM instance
    Structured output that works with every vendor.

    1. Put the JSON Schema of `schema` (a Pydantic model) into the system prompt.
    2. Ask for JSON (using the vendor's JSON mode when it has one).
    3. Parse and validate the reply with Pydantic.
    4. If validation fails, show the model its own error and ask again
       (a "repair loop"), up to `max_repairs` times.
    Returns an instance of `schema`, or raises ExtractionError.

['reg_id', 'full_name', 'email', 'email_valid', 'phone', 'company', 'seniority', 'state', 'registration_date',
'ticket_type', 'amount_paid', 'dietary_category', 'workshop', 'duplicate_of']
  reg_id       full_name                            email email_valid         phone                        compa
ny               seniority state registration_date ticket_type amount_paid dietary_category           workshop d
uplicate_of
0  R5068     Kara O'Neil           kara.oneil@outlook.com        True  +17018926765                        Cargi
```

```
ll               Executive    MN        2026-01-15  Early Bird       249.0             None   Dashboard Design
1  R5090     Nikhil Wold          nikhil_wold@cargill.org       True  +16058848469                       Cargi
ll  Individual Contributor    SD        2026-02-01  Early Bird       249.0             None   Dashboard Design
2  R5030    Emily Dvorak          emilyd@citibanksout.org       True  +15158384412          Citibank South Dako
ta                 Manager    IA        2026-03-08     Regular       349.0             None  LLMs for Analysts
3  R5012  Ximena Solberg  ximena.solberg@southdakotas.org       True                South Dakota State Universi
ty                 Student    NE        2026-01-30     Student        49.0             None   Causal Inference
4  R5027   Yusuf Iverson            yusufi@daktronics.org       True  +17017977864                    Daktroni
cs  Individual Contributor    ND        2026-01-20  Early Bird       249.0      Gluten-Free              None

================================================== state
state
South Dakota    16
S Dakota        11
minnesota        7
IOWA             7
south dakota     7
SD               7
S.D.             6
SoDak            6
ND               5
IA               5
Iowa             4
North Dakota     4
Minn.            4
CO               4
Minnesota        4
Colorado         3
NE               3
N.D.             3
Ia.              3
Neb              3
Wisc.            2
MN               1
Colo.            1
Nebr.            1
Wisconsin        1
N. Dakota        1
WI               1

================================================== ticket_type
ticket_type
Regular                   16
Standard                  16
regular admission         11
EARLYBIRD                 11
Student                   10
early-bird                 8
Early Bird                 7
REGULAR                    7
STUDENT                    7
EB                         7
SPEAKER                    6
Early bird registration    6
speaker (comp)             4
student pass               3
Speaker                    1

================================================== amount_paid
amount_paid
USD 349    14
349        13
349.00     12
249.00      9
$249.00     8
USD 249     8
```

```
$349        7
$49.00      7
249         7
$49         6
$249        5
$349.00     4
$0.00       4
49.00       4
free        3
<blank>     2
comp        2
0           2
USD 49      2
49          1

================================================== dietary_notes
dietary_notes
<blank>                                                   29
none                                                      22
no restrictions                                           17
-                                                         12
lacto-ovo vegetarian                                       7
Vegetarian (no meat, fish ok?) actually no fish either     4
veggie please                                              4
vegan                                                      4
Halal meals only                                           4
GF                                                         3
no nuts!!                                                  3
vegetarian                                                 3
peanut allergy                                             2
halal                                                      2
no gluten please                                           1
kosher                                                     1
celiac - strict GF                                         1
gluten free                                                1

================================================== workshop
workshop
Causal Inference     26
dbt Fundamentals     25
LLMs for Analysts    23
Dashboard Design     21
<blank>              18
none                  7

================================================== job_title
job_title
Project Manager - Data               10
Machine Learning Engineer            10
Undergraduate student                 9
Sr. Data Analyst                      7
BI Developer                          7
Business Analyst                      6
Dir. Data Strategy                    6
Analytics Consultant                  6
Mgr, Business Intelligence            6
Senior Director, Enterprise Data      5
Director of Analytics                 5
Reporting Specialist                  5
Graduate Assistant                    5
Data Science Team Lead                4
Statistician                          3
MS Business Analytics Student         3
Chief Data Officer                    3
PhD Candidate, Information Systems    3
Chief Analytics Officer               3
```

```
Data Analyst                          3
Supervisor, Reporting                 3
Data Engineer II                      2
SVP Technology                        2
Analytics Manager                     2
Head of BI                            1
CIO                                   1

================================================== company
company
South Dakota State University    12
Cargill                          11
Midco                            10
University of South Dakota        9
Daktronics                        7
UNIVERSITY OF SOUTH DAKOTA        7
Hy-Vee                            6
Wells Fargo                       6
Raven Industries                  5
Citibank South Dakota             4
SANFORD HEALTH                    4
Target                            4
DAKTRONICS                        4
Sanford Health                    4
WELLS FARGO                       3
Dakota State University           3
TARGET                            2
Black Hills Energy                2
Great Western Bank                2
AVERA                             2
GREAT WESTERN BANK                2
CARGILL                           2
POET                              2
SOUTH DAKOTA STATE UNIVERSITY     1
CITIBANK SOUTH DAKOTA             1
BLACK HILLS ENERGY                1
Avera                             1
RAVEN INDUSTRIES                  1
MIDCO                             1
Mayo Clinic                       1
             full_name                           email            phone  registration_date
95      HANNAH MAHMOUD        hmahmoud@citibanksout.org     612.354.8876         2026/03/28
54    Carmen Lundquist         carmenl@universityof.org       5157197512        14-Feb-2026
59          YUSUF VANG                   yvanggmail.com  +1 605 737 1435  February 12, 2026
117         Yoder, Zoe                zyoder@yahoo.com  +1 608 940 4210         2026/01/17
77     Samuel Castillo        samuel.castillo@gmail.com              NaN         01/30/2026
46    Gunnar Zimmerman  gunnar_zimmerman@wellsfargo.org  +1 515 395 9323   January 15, 2026
31    Anderson, Brooke        BROOKE_ANDERSON@GMAIL.COM      13033886303         02/13/2026
81        Aaliyah Vang          AALIYAH.VANG@TARGET.ORG       6129653184         03/27/2026
48      Brooke Iverson               biverson@yahoo.com  +1 605 256 6129        28-Jan-2026
98        Jamal Nordby         jamal_nordby@outlook.com     16082045716     March 19, 2026
97   Gutierrez, Samuel                 sgutierrez@midco              NaN         2026-03-24
99           tara eide                  tarae@yahoo.com  +1 605 407 8892         02/24/2026
2         Emily Dvorak          emilyd@citibanksout.org   (515) 838-4412         2026-03-08
92      Olga Jorgensen    olga_jorgensen@daktronics.org     605-780-9729         2026/02/06
113      HANNAH O'NEIL               honeil@outlook.com      16083983485      March 9, 2026
56        Carmen Quist           carmenq@daktronics.org   (605) 996-7404         03/05/2026
106     samuel iverson       samuel_iverson@cargill.org              NaN        22-Feb-2026
103      Samuel Nordby            samuel_nordby@cargill     605.537.6834            3/17/26
67          Ben Haugen           bhaugen@daktronics.org              NaN         2026-01-20
62         maya nordby         mnordby@citibanksout.org     605.612.2396         2026/04/12
112         quinn fong                  qfong@yahoo.com     701.529.0909         2026/02/06
17      Isabel O'Neil           isabel_oneil@target.org     605.822.7294         04/19/2026
83          Cody Quist                 cquist@gmail.com       3035456034         2026/03/25
108    Rachel Kowalski      rachel_kowalski@cargill.org  +1 605 855 6634             2/9/26
73        Vanessa Thao               vanessat@yahoo.com       6053937832     March 13, 2026
```

In [8]:

```
# Part C: cleaner

from dateutil import parser as dateparser

# Text helper: stripped string, blanks as ""
def txt(x):
    return "" if pd.isna(x) else str(x).strip()

# Name: "SMITH, Ann" -> "Ann Smith", title case keeps O'Neil
def clean_name(x):
    x = txt(x)
    if "," in x:
        last, first = [p.strip() for p in x.split(",", 1)]
        x = f"{first} {last}"
    return " ".join(x.split()).title()

# Email: lowercase, then check one @, a name, and a dotted domain
def clean_email(x):
    e = txt(x).lower()
    ok = bool(re.fullmatch(r"[^@\s]+@[^@\s]+\.[^@\s]+", e)) and not e.endswith(".")
    return (e if ok else ""), ok

# Phone: keep digits, drop a leading 1, format as +1 and 10 digits
def clean_phone(x):
    d = re.sub(r"\D", "", txt(x))
    if len(d) == 11 and d.startswith("1"):
        d = d[1:]
    return "+1" + d if len(d) == 10 else ""

# Company: most frequent spelling that is not ALL CAPS
comp_counts = raw["company"].dropna().str.strip().value_counts()
company_map = {}
for key in comp_counts.index.str.lower().unique():
    spellings = comp_counts[comp_counts.index.str.lower() == key]
    normal = spellings[~spellings.index.str.isupper()]
    company_map[key] = normal.index[0] if len(normal) > 0 else spellings.index[0]

def clean_company(x):
    return company_map.get(txt(x).lower(), txt(x))

# State: map every spelling to its two-letter code
state_map = {
    "sd": "SD", "southdakota": "SD", "sdakota": "SD", "sodak": "SD",
    "nd": "ND", "northdakota": "ND", "ndakota": "ND",
    "mn": "MN", "minnesota": "MN", "minn": "MN",
    "ia": "IA", "iowa": "IA",
    "ne": "NE", "nebraska": "NE", "neb": "NE", "nebr": "NE",
    "wi": "WI", "wisconsin": "WI", "wisc": "WI",
    "co": "CO", "colorado": "CO", "colo": "CO",
}

def clean_state(x):
    key = re.sub(r"[\s.]", "", txt(x).lower())
    return state_map.get(key, "")

# Date: year-first if it starts with 4 digits, otherwise US month-first
def clean_date(x):
    s = txt(x)
    if s == "":
        return ""
    try:
        yearfirst = bool(re.match(r"^\d{4}", s))
        return dateparser.parse(s, yearfirst=yearfirst, dayfirst=False).strftime("%Y-%m-%d")
    except (ValueError, OverflowError):
        return ""
```

```python
# Ticket type: map labels by keyword
def clean_ticket(x):
    s = txt(x).lower()
    if "early" in s or s == "eb":
        return "Early Bird"
    if "student" in s:
        return "Student"
    if "speaker" in s:
        return "Speaker"
    if "regular" in s or "standard" in s:
        return "Regular"
    return ""

# Amount: number in dollars, free/comp = 0, blank = missing
def clean_amount(x):
    s = txt(x).lower()
    if s == "":
        return np.nan
    if s in ("free", "comp"):
        return 0.0
    d = re.sub(r"[^\d.]", "", s)
    return float(d) if d else np.nan

# Workshop: standard names, blank or "none" = None
workshop_map = {w.lower(): w for w in
                ["LLMs for Analysts", "dbt Fundamentals", "Causal Inference", "Dashboard Design"]}

def clean_workshop(x):
    return workshop_map.get(txt(x).lower(), "None")

# Small contracts for the AI calls
class SeniorityOut(BaseModel):
    seniority: Seniority

class DietOut(BaseModel):
    dietary_category: Diet

seniority_instructions = (
    "Classify the job title into one seniority level. "
    "Student: students, graduate assistants, PhD candidates. "
    "Manager: managers, team leads, supervisors, project managers. "
    "Director: directors (including senior directors) and heads of a function. "
    "Executive: C-level (Chief ..., CIO, CDO), VP, SVP. "
    "Individual Contributor: everyone else, including senior analysts, engineers, consultants, specialists."
)

diet_instructions = (
    "Classify the dietary note into one category: None, Vegetarian, Vegan, Gluten-Free, Halal, Kosher, Nut
Allergy. "
    "Lacto-ovo and 'veggie' mean Vegetarian. Celiac, GF, or no gluten mean Gluten-Free. "
    "No nuts or peanut allergy mean Nut Allergy."
)

# Keyword fallbacks, used only if an AI call fails
def seniority_rule(t):
    t = t.lower()
    if any(k in t for k in ["student", "graduate assistant", "phd"]):
        return "Student"
    if any(k in t for k in ["chief", "cio", "svp", "vp "]):
        return "Executive"
    if "director" in t or "dir." in t or t.startswith("head of"):
        return "Director"
    if any(k in t for k in ["manager", "mgr", "lead", "supervisor"]):
        return "Manager"
    return "Individual Contributor"
```

```
def diet_rule(t):
    t = t.lower()
    if "vegan" in t:
        return "Vegan"
    if "veg" in t:
        return "Vegetarian"
    if "gluten" in t or "gf" in t or "celiac" in t:
        return "Gluten-Free"
    if "halal" in t:
        return "Halal"
    if "kosher" in t:
        return "Kosher"
    if "nut" in t:
        return "Nut Allergy"
    return "None"

# Dietary values clear enough for a rule (no AI needed)
diet_simple = {"": "None", "none": "None", "-": "None", "no restrictions": "None",
               "vegan": "Vegan", "vegetarian": "Vegetarian", "halal": "Halal", "kosher": "Kosher"}

# AI step: one call per unique value, with failure handling
ai_calls, ai_failures = 0, 0
ai_log = []

def ask_ai(value, schema, instructions, field, fallback):
    global ai_calls, ai_failures
    ai_calls += 1
    try:
        out = getattr(llm.extract(value, schema, instructions=instructions), field)
        source = "AI"
    except Exception as e:
        out = fallback(value)
        ai_failures += 1
        source = f"rule fallback ({type(e).__name__})"
    ai_log.append({"field": field, "input": value, "output": out, "source": source})
    return out

seniority_lookup = {}
for t in raw["job_title"].fillna("").str.strip().unique():
    seniority_lookup[t] = ask_ai(t, SeniorityOut, seniority_instructions, "seniority", seniority_rule)

diet_lookup = {}
for d in raw["dietary_notes"].fillna("").str.strip().unique():
    if d.lower() in diet_simple:
        diet_lookup[d] = diet_simple[d.lower()]
    else:
        diet_lookup[d] = ask_ai(d, DietOut, diet_instructions, "dietary_category", diet_rule)

# Build the clean table, one row per raw registration
rows = []
for _, r in raw.iterrows():
    email_clean, email_ok = clean_email(r["email"])
    rows.append({
        "reg_id": txt(r["reg_id"]),
        "full_name": clean_name(r["full_name"]),
        "email": email_clean,
        "email_valid": email_ok,
        "phone": clean_phone(r["phone"]),
        "company": clean_company(r["company"]),
        "seniority": seniority_lookup[txt(r["job_title"])],
        "state": clean_state(r["state"]),
        "registration_date": clean_date(r["registration_date"]),
        "ticket_type": clean_ticket(r["ticket_type"]),
        "amount_paid": clean_amount(r["amount_paid"]),
        "dietary_category": diet_lookup[txt(r["dietary_notes"])],
```

```
        "workshop": clean_workshop(r["workshop"]),
    })
clean = pd.DataFrame(rows)

# Validate every row; failures go to a review list, not dropped
review = []
for rec in clean.to_dict("records"):
    rec_v = dict(rec)
    rec_v["amount_paid"] = None if pd.isna(rec_v["amount_paid"]) else rec_v["amount_paid"]
    rec_v["email"] = rec_v["email"] or None
    rec_v["phone"] = rec_v["phone"] or None
    try:
        CleanRegistration(**rec_v)
    except ValidationError as err:
        msgs = "; ".join(f"{'.'.join(map(str, e['loc'])) or 'record'}: {e['msg']}" for e in err.errors())
        review.append({"reg_id": rec["reg_id"], "errors": msgs})
review = pd.DataFrame(review, columns=["reg_id", "errors"])

# Report
print("Clean rows:", len(clean))
print("AI calls:", ai_calls, "| AI failures (rule fallback used):", ai_failures)
print("Rows passing the contract:", len(clean) - len(review), "| Rows sent to review:", len(review))
display(pd.DataFrame(ai_log))
display(review)
clean.head(10)
```

```
[llmkit] ServerError (attempt 1); waiting 2s before retrying.
[llmkit] ServerError (attempt 1); waiting 2s before retrying.
Clean rows: 120
AI calls: 36 | AI failures (rule fallback used): 0
Rows passing the contract: 120 | Rows sent to review: 0
```

	field	input	output	source
0	seniority	Chief Data Officer	Executive	AI
1	seniority	Business Analyst	Individual Contributor	AI
2	seniority	Project Manager - Data	Manager	AI
3	seniority	MS Business Analytics Student	Student	AI
4	seniority	Reporting Specialist	Individual Contributor	AI
5	seniority	Undergraduate student	Student	AI
6	seniority	Director of Analytics	Director	AI
7	seniority	Data Science Team Lead	Manager	AI
8	seniority	Senior Director, Enterprise Data	Director	AI
9	seniority	Data Engineer II	Individual Contributor	AI
10	seniority	Sr. Data Analyst	Individual Contributor	AI
11	seniority	Dir. Data Strategy	Director	AI
12	seniority	Head of BI	Director	AI
13	seniority	Analytics Manager	Manager	AI
14	seniority	Graduate Assistant	Student	AI
15	seniority	Analytics Consultant	Individual Contributor	AI
16	seniority	Machine Learning Engineer	Individual Contributor	AI
17	seniority	BI Developer	Individual Contributor	AI
18	seniority	Mgr, Business Intelligence	Manager	AI
19	seniority	PhD Candidate, Information Systems	Student	AI
20	seniority	SVP Technology	Executive	AI
21	seniority	Chief Analytics Officer	Executive	AI
22	seniority	Supervisor, Reporting	Manager	AI
23	seniority	Data Analyst	Individual Contributor	AI

24	seniority	Statistician	Individual Contributor	AI
25	seniority	CIO	Executive	AI
26	dietary_category	GF	Gluten-Free	AI
27	dietary_category	lacto-ovo vegetarian	Vegetarian	AI
28	dietary_category	Vegetarian (no meat, fish ok?) actually no fis...	Vegetarian	AI
29	dietary_category	veggie please	Vegetarian	AI
30	dietary_category	peanut allergy	Nut Allergy	AI
31	dietary_category	no nuts!!	Nut Allergy	AI
32	dietary_category	no gluten please	Gluten-Free	AI
33	dietary_category	Halal meals only	Halal	AI
34	dietary_category	celiac - strict GF	Gluten-Free	AI
35	dietary_category	gluten free	Gluten-Free	AI

reg_id errors

Out[8]:

	reg_id	full_name	email	email_valid	phone	company	seniority	state	registration_date	ticket_type	amount_paid	dietary_category	workshop
0	R5068	Kara O'Neil	kara.oneil@outlook.com	True	+17018926765	Cargill	Executive	MN	2026-01-15	Early Bird	249.0	None	Dashboard Design
1	R5090	Nikhil Wold	nikhil_wold@cargill.org	True	+16058848469	Cargill	Individual Contributor	SD	2026-02-01	Early Bird	249.0	None	Dashboard Design
2	R5030	Emily Dvorak	emilyd@citibanksout.org	True	+15158384412	Citibank South Dakota	Manager	IA	2026-03-08	Regular	349.0	None	LLMs for Analysts
3	R5012	Ximena Solberg	ximena.solberg@southdakotas.org	True		South Dakota State University	Student	NE	2026-01-30	Student	49.0	None	Causal Inference
4	R5027	Yusuf Iverson	yusufi@daktronics.org	True	+17017977864	Daktronics	Individual Contributor	ND	2026-01-20	Early Bird	249.0	Gluten-Free	None
5	R5015	Kara Yoder	kyoder@southdakotas.org	True	+16056717510	South Dakota State University	Student	SD	2026-01-19	Student	49.0	Vegetarian	Dashboard Design
6	R5095	Pedro Quist	pquist@sanfordhealt.org	True	+16058991996	Sanford Health	Director	SD	2026-01-23	Regular	349.0	Vegetarian	LLMs for Analysts
7	R5043	Elena Eide	elena_eide@yahoo.com	True	+16055525270	Sanford Health	Manager	SD	2026-03-20	Regular	349.0	Vegetarian	Causal Inference
8	R5050	Lars Wold	larsw@yahoo.com	True	+16058564913	South Dakota State University	Student	SD	2026-03-14	Student	49.0	None	LLMs for Analysts
9	R5060	Carmen Kowalski	carmen.kowalski@ravenindustr.org	True	+16084966935	Raven Industries	Individual Contributor	IA	2026-02-19	Early Bird	NaN	None	Dashboard Design

In [9]:

```
# Part C: evaluation on the dev sample (field accuracy table)

# Columns to score (duplicate_of is Part D)
eval_cols = ["full_name", "email", "email_valid", "phone", "company", "seniority", "state",
             "registration_date", "ticket_type", "amount_paid", "dietary_category", "workshop"]

# Put values in the same text form so the comparison is fair
def norm(col, x):
    if pd.isna(x) or str(x).strip() in ("", "nan", "NaN"):
        return ""
    x = str(x).strip()
    if col == "amount_paid":
        return f"{float(x):.2f}"
    if col == "email_valid":
        return "True" if x.lower() == "true" else "False"
    return x

# Match cleaned rows to the gold rows by reg_id
merged = dev_gold.merge(clean, on="reg_id", suffixes=("_gold", "_pred"), how="left")
```

```
# Accuracy per column, and a list of every mismatch
acc_rows, mismatches = [], []
for c in eval_cols:
    g = merged[c + "_gold"].apply(lambda x: norm(c, x))
    p = merged[c + "_pred"].apply(lambda x: norm(c, x))
    acc_rows.append({"column": c, "correct": int((g == p).sum()), "accuracy": round((g == p).mean() * 100,
1)})
    for i in merged.index[g != p]:
        mismatches.append({"reg_id": merged.loc[i, "reg_id"], "column": c, "gold": g[i], "predicted": p[i]})

acc_table = pd.DataFrame(acc_rows)
mismatches = pd.DataFrame(mismatches, columns=["reg_id", "column", "gold", "predicted"])

print("Dev rows:", len(dev_gold))
print("Overall field accuracy:", round(acc_table["accuracy"].mean(), 1), "%")
display(acc_table)
display(mismatches)
```

```
Dev rows: 25
Overall field accuracy: 100.0 %
```

	column	correct	accuracy
0	full_name	25	100.0
1	email	25	100.0
2	email_valid	25	100.0
3	phone	25	100.0
4	company	25	100.0
5	seniority	25	100.0
6	state	25	100.0
7	registration_date	25	100.0
8	ticket_type	25	100.0
9	amount_paid	25	100.0
10	dietary_category	25	100.0
11	workshop	25	100.0

reg_id	column	gold	predicted

Fixing the largest error sources

On the dev sample the cleaner reached 100% field accuracy, so there were no remaining error sources to fix. The main risks were handled up front by checking every distinct raw value before writing the rules:

- Dates: extra formats such as `14-Feb-2026` and `3/17/26`
- States: abbreviations such as `SoDak`, `Nebr.`, and `Wisc.`
- Ticket types: labels such as `Standard` and `EB`
- Job titles: tricky cases such as `Sr. Data Analyst` (Individual Contributor) and `Senior Director` (Director)

The AI log was also reviewed by hand, and all 36 AI decisions were correct.

Part D. Duplicates (15 points)

Find repeat registrations. Generate candidates with deterministic keys (for example normalized email, phone, or last name plus company), verify each candidate with an explicit rule (or an LLM decision that code then checks),

and keep an audit log (kept id, dropped id, evidence, verified yes or no). Fill the `duplicate_of` column and report accuracy on dev. Session 3, 0.6 shows normalized keys and `duplicated(keep=False)`; 4.5 builds candidates from a key, verifies proposals with code, and logs every decision.

In [12]:
```
# Part D: duplicate detection with verification and an audit log
from difflib import SequenceMatcher
from itertools import combinations

# Number part of a reg_id, so R5012 sorts before R5068
def id_num(r):
    return int(r[1:])

# Name helpers: lowercase letters only (O'Neil and ONeil match)
def name_norm(x):
    return re.sub(r"[^a-z ]", "", str(x).lower()).strip()

def first_name(x):
    parts = name_norm(x).split()
    return parts[0] if parts else ""

def last_name(x):
    parts = name_norm(x).split()
    return parts[-1] if parts else ""

# First names agree if equal, or if one is an initial of the other (J. = Jamal)
def first_names_agree(a, b):
    if a == b:
        return True
    if len(a) == 1 or len(b) == 1:
        return a[0] == b[0]
    return False

# Table used for matching
d = clean[["reg_id", "full_name", "phone", "company"]].copy()
d["email_key"] = raw["email"].fillna("").str.strip().str.lower()
d["name_key"] = d["full_name"].apply(name_norm)
d["last_company_key"] = d["full_name"].apply(last_name) + "|" + d["company"].str.lower()
d = d.set_index("reg_id")

# Step 1: candidate pairs from deterministic keys
keys = {"email": "email_key", "phone": "phone", "last name + company": "last_company_key"}
candidates = {}
for key_name, col_name in keys.items():
    groups = d[d[col_name] != ""].groupby(col_name).groups
    for _, ids in groups.items():
        for a, b in combinations(sorted(ids, key=id_num), 2):
            candidates.setdefault((a, b), []).append(key_name)

# Step 2: verify each candidate with an explicit rule
audit = []
for (a, b), matched_keys in candidates.items():
    ra, rb = d.loc[a], d.loc[b]
    sim = SequenceMatcher(None, ra["name_key"], rb["name_key"]).ratio()
    same_email = ra["email_key"] != "" and ra["email_key"] == rb["email_key"]
    same_phone = ra["phone"] != "" and ra["phone"] == rb["phone"]
    same_company = ra["company"].lower() == rb["company"].lower()
    same_last = last_name(ra["full_name"]) == last_name(rb["full_name"])
    same_first = first_names_agree(first_name(ra["full_name"]), first_name(rb["full_name"]))
    verified = same_last and same_first and (same_email or same_phone or same_company)
    audit.append({
        "kept_id": a,
        "dropped_id": b,
        "keys_matched": ", ".join(matched_keys),
        "evidence": (f"names '{ra['full_name']}' vs '{rb['full_name']}' (similarity {sim:.2f}); "
                     f"same first={same_first}, same last={same_last}, same email={same_email}, "
```

```
                    f"same phone={same_phone}, same company={same_company}"),
        "verified": "yes" if verified else "no",
    })
audit_log = pd.DataFrame(audit, columns=["kept_id", "dropped_id", "keys_matched", "evidence", "verified"])

# Step 3: group verified pairs so each person has one first registration
parent = {r: r for r in d.index}
def find(r):
    while parent[r] != r:
        r = parent[r]
    return r
for _, row in audit_log[audit_log["verified"] == "yes"].iterrows():
    ra, rb = find(row["kept_id"]), find(row["dropped_id"])
    if ra != rb:
        keep, drop = sorted([ra, rb], key=id_num)
        parent[drop] = keep

# Fill duplicate_of with the lowest reg_id of the person's group
clean["duplicate_of"] = clean["reg_id"].apply(lambda r: find(r) if find(r) != r else "")

# Accuracy on the dev sample
dup_check = dev_gold[["reg_id", "duplicate_of"]].merge(
    clean[["reg_id", "duplicate_of"]], on="reg_id", suffixes=("_gold", "_pred"))
dup_check["match"] = dup_check["duplicate_of_gold"].fillna("") == dup_check["duplicate_of_pred"]

# Report
print("Candidate pairs:", len(audit_log))
print("Verified duplicates (yes):", (audit_log["verified"] == "yes").sum(),
      "| Rejected (no):", (audit_log["verified"] == "no").sum())
print("Rows marked as repeat registrations:", (clean["duplicate_of"] != "").sum())
print("duplicate_of accuracy on dev:", round(dup_check["match"].mean() * 100, 1), "%")
pd.set_option("display.max_colwidth", None)
display(audit_log.sort_values("verified", ascending=False))
display(dup_check[~dup_check["match"]])
```

```
Candidate pairs: 22
Verified duplicates (yes): 10 | Rejected (no): 12
Rows marked as repeat registrations: 10
duplicate_of accuracy on dev: 100.0 %
```

	kept_id	dropped_id	keys_matched	evidence	verified
0	R5084	R5113	email, phone, last name + company	names 'Dakota Jorgensen' vs 'Dakota Jorgensen' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
2	R5101	R5118	email, phone, last name + company	names 'Farid Fong' vs 'Farid Fong' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
3	R5070	R5114	email, phone, last name + company	names 'Gunnar Haugen' vs 'Gunnar Haugen' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
4	R5078	R5115	email, phone, last name + company	names 'Hiro Reyes' vs 'Hiro Reyes' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
5	R5021	R5111	email, phone, last name + company	names 'Ingrid Zimmerman' vs 'Ingrid Zimmerman' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
6	R5053	R5116	email, last name + company	names 'Jamal Ueland' vs 'J. Ueland' (similarity 0.80); same first=True, same last=True, same email=True, same phone=False, same company=True	yes
7	R5088	R5112	email, last name + company	names 'Pedro Mahmoud' vs 'Pedro Mahmoud' (similarity 1.00); same first=True, same last=True, same email=True, same phone=False, same company=True	yes
8	R5037	R5117	email, phone, last name + company	names 'Tara Eide' vs 'Tara Eide' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
9	R5005	R5119	email, phone, last name + company	names 'Wes Bergstrom' vs 'Wes Bergstrom' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
1	R5030	R5120	email, phone, last name + company	names 'Emily Dvorak' vs 'Emily Dvorak' (similarity 1.00); same first=True, same last=True, same email=True, same phone=True, same company=True	yes
16	R5032	R5085	last name + company	names 'Farid O'Neil' vs 'Hannah O'Neil' (similarity 0.61); same first=False, same last=True, same email=False, same phone=False, same company=True	no

20	R5019	R5061	last name + company	names 'Kara Vang' vs 'Aaliyah Vang' (similarity 0.67); same first=False, same last=True, same email=False, same phone=False, same company=True	no
19	R5064	R5116	last name + company	names 'Emily Ueland' vs 'J. Ueland' (similarity 0.70); same first=False, same last=True, same email=False, same phone=False, same company=True	no
18	R5053	R5064	last name + company	names 'Jamal Ueland' vs 'Emily Ueland' (similarity 0.75); same first=False, same last=True, same email=False, same phone=False, same company=True	no
17	R5076	R5077	last name + company	names 'Dakota Reyes' vs 'Yusuf Reyes' (similarity 0.52); same first=False, same last=True, same email=False, same phone=False, same company=True	no
11	R5074	R5093	last name + company	names 'Dakota Iverson' vs 'Brooke Iverson' (similarity 0.64); same first=False, same last=True, same email=False, same phone=False, same company=True	no
15	R5001	R5017	last name + company	names 'Rachel Nordby' vs 'Samuel Nordby' (similarity 0.77); same first=False, same last=True, same email=False, same phone=False, same company=True	no
14	R5097	R5102	last name + company	names 'Ingrid Lundquist' vs 'Yusuf Lundquist' (similarity 0.65); same first=False, same last=True, same email=False, same phone=False, same company=True	no
13	R5036	R5092	last name + company	names 'Lars Jorgensen' vs 'Brooke Jorgensen' (similarity 0.73); same first=False, same last=True, same email=False, same phone=False, same company=True	no
12	R5014	R5083	last name + company	names 'Ximena Jorgensen' vs 'Maya Jorgensen' (similarity 0.80); same first=False, same last=True, same email=False, same phone=False, same company=True	no
10	R5027	R5082	last name + company	names 'Yusuf Iverson' vs 'Lars Iverson' (similarity 0.72); same first=False, same last=True, same email=False, same phone=False, same company=True	no
21	R5003	R5028	last name + company	names 'Rachel Wold' vs 'Grace Wold' (similarity 0.86); same first=False, same last=True, same email=False, same phone=False, same company=True	no

reg_id	duplicate_of_gold	duplicate_of_pred	match

Part E. Deliverables (20 points)

1. Save `hw3_clean_registrations.csv` with columns `reg_id, full_name, email, email_valid, phone, company, seniority, state, registration_date, ticket_type, amount_paid, dietary_category, workshop, duplicate_of`.
2. A before/after quality report (a small table) using your Part A checks.
3. Reflection (200 words): which fields should AI own, which should code own, and which need a human? Use your accuracy numbers and the number of AI calls.

In [13]:
```python
# Part E: save the output file and build the before/after quality report

# Column order required by the assignment
out_cols = ["reg_id", "full_name", "email", "email_valid", "phone", "company", "seniority", "state",
            "registration_date", "ticket_type", "amount_paid", "dietary_category", "workshop", "duplicate_of"]

# Save the clean file
output = clean[out_cols].copy()
output.to_csv("hw3_clean_registrations.csv", index=False)

# Read it back to confirm the file is well formed
check_file = pd.read_csv("hw3_clean_registrations.csv", dtype=str, keep_default_na=False)
print("Saved file shape:", check_file.shape)
print("Columns match:", list(check_file.columns) == out_cols)
print("Unique reg_ids:", check_file["reg_id"].nunique())

# Same Part A checks as a function, so they can run on raw and clean data
def profile(df, title_col, diet_col, resolved_mask=None):
    s = lambda c: df[c].fillna("").astype(str).str.strip()
    name, email, phone, company = s("full_name"), s("email").str.lower(), s("phone"), s("company")
    state, date, ticket, amount = s("state"), s("registration_date"), s("ticket_type"), s("amount_paid")
    diet, workshop = s(diet_col), s("workshop")
```

```
    # Only count duplicates that were not resolved (after cleaning)
    active = resolved_mask if resolved_mask is not None else pd.Series(True, index=df.index)

    n_spell = company.groupby(company.str.lower()).transform("nunique")
    name_key = name.str.lower().str.replace(",", " ").str.split().apply(lambda w: " ".join(sorted(w)))
    amt, price = amount.apply(amount_num), ticket.apply(ticket_price)

    return pd.DataFrame({
        "email missing": email == "",
        "phone missing": phone == "",
        "amount_paid missing": amount == "",
        "job_title missing": s(title_col) == "",
        "email invalid format": (email != "") & ~(email.str.match(email_re) & ~email.str.endswith(".")),
        "phone not 10 digits": (phone != "") & ~phone.apply(phone_ok),
        "state not a valid 2-letter code": ~state.isin(valid_states),
        "date not YYYY-MM-DD": ~date.str.match(iso_date_re),
        "name not in First Last title case": name.str.contains(",") | name.str.isupper() | name.str.islower(),
        "company has several spellings": (company != "") & (n_spell > 1),
        "ticket_type not a standard label": ~ticket.isin(std_tickets),
        "dietary_notes not a standard category": ~diet.isin(std_diets),
        "workshop not a standard name": (workshop != "") & ~workshop.isin(std_workshops),
        "duplicate email": active & (email != "") & email[active].duplicated(keep=False).reindex(df.index,
fill_value=False),
        "duplicate person name": active & name_key[active].duplicated(keep=False).reindex(df.index,
fill_value=False),
        "amount does not match ticket price": amt.notna() & price.notna() & (amt != price),
    })

# Run the checks before and after cleaning
before = profile(raw, "job_title", "dietary_notes")
after = profile(output, "seniority", "dietary_category", resolved_mask=(output["duplicate_of"] == ""))

# Before/after report table
report = pd.DataFrame({
    "dimension": [dimension[c] for c in before.columns],
    "issue": before.columns,
    "before": before.sum().values,
    "after": after.sum().values,
})
report = report.sort_values(["dimension", "before"], ascending=[True, False]).reset_index(drop=True)

print("\nRows with at least one issue - before:", before.any(axis=1).sum(), "| after:",
after.any(axis=1).sum())
print("AI calls used:", ai_calls, "| Repeat registrations flagged:", (output["duplicate_of"] != "").sum())
report
```

```
Saved file shape: (120, 14)
Columns match: True
Unique reg_ids: 120

Rows with at least one issue - before: 120 | after: 25
AI calls used: 36 | Repeat registrations flagged: 10
```

Out[13]:

	dimension	issue	before	after
0	Accuracy	amount does not match ticket price	0	0
1	Completeness	phone missing	17	17
2	Completeness	amount_paid missing	2	2
3	Completeness	email missing	0	9
4	Completeness	job_title missing	0	0
5	Consistency	dietary_notes not a standard category	120	0
6	Consistency	company has several spellings	108	0
7	Consistency	date not YYYY-MM-DD	96	0

8	Consistency	ticket_type not a standard label	86	0
9	Consistency	name not in First Last title case	42	0
10	Consistency	workshop not a standard name	7	0
11	Uniqueness	duplicate email	20	0
12	Uniqueness	duplicate person name	18	0
13	Validity	state not a valid 2-letter code	94	0
14	Validity	email invalid format	9	0
15	Validity	phone not 10 digits	0	0

Reflection:

Code should own the fields with clear format rules: name, email, phone, company, state, date, ticket type, amount, and workshop. Rules handled all nine with 100% accuracy on the 25 dev rows, used zero AI calls, run in under a second, and give the same answer every time. Using AI here would add cost and risk with no gain.

AI should own the fields that need judgment about meaning: seniority from job titles and dietary category from free-text notes. Rules would need a long list of exceptions (for example, "Sr. Data Analyst" is an Individual Contributor, but "Senior Director" is a Director). By calling the AI once per unique value instead of once per row, I used 36 AI calls instead of 240. All 36 answers were correct, and both fields scored 100% on dev. The contract still checked every AI answer against the allowed values, and the AI log kept each decision visible.

A human is needed where the data cannot answer the question. The 9 invalid emails and 17 missing phones need someone to contact those attendees. The 2 missing amounts need checking against payment records for revenue reconciliation. Borderline duplicates, such as "J. Ueland" matched to "Jamal Ueland" by initial, should be confirmed by a person before badges are printed.

Make your submission PDF (run this last)

Save your work first (File > Save), then run the cell below. It turns the whole notebook, outputs included, into `HW3_submission.pdf` and downloads it, plus an HTML copy as a backup. The first run takes one to two minutes because it installs a PDF printer.

Do not use File > Print in Colab: it prints only what is laid out on screen, so long notebooks are cut off partway down a page. If the PDF step fails, open the downloaded `HW3_submission.html` in your browser and print that page to PDF (Ctrl+P or Cmd+P, then *Save as PDF*); a standalone page prints in full.

In [ ]:
```
#@title Make the PDF and HTML copies of this notebook
# code hidden in this copy
```